Gebze Technical University

CSE 312 – Semester Project

Hypothetical CPU and Operating System

Name: MUHAMMED EMİR KARA

Student ID: 210104004071

Table Of Contents

1. Introduction **3**
1.1. Background 3
1.2. Literature Review 4
1.3. Objectives 6
2. METHODOLOGY **8**
2.1. Simulation Environment and Hypothetical CPU 8
2.1.1. Overview of the Simulation 8
2.1.2. The GTU-C312 Hypothetical CPU Architecture 9
2.1.3. Program Loading and Format 15
2.2. Operating System (GTU-OS) 21
2.2.1. Core Data Structures and Memory Layout 22
2.2.2. OS Initialization, Thread Table Setup, and Main Loop 23
2.2.3. Scheduling Algorithm (Round-Robin with Block Handling) 26
2.2.4. Context Switching 27
2.2.5. System Call Handling 30
2.2.6. Overall Operational Flow 31
2.2.7. User Thread Implementation 31
3. Results and Discussion **33**
4. Conclusion **48**
5. References **50**

1. Introduction

The Operating System (OS) stands as a cornerstone of modern computing, acting as the intermediary between computer hardware and user applications. It manages system resources, provides a consistent environment for software execution, and offers essential services that simplify complex hardware interactions. This project undertakes the ambitious task of designing and implementing a rudimentary operating system, named GTU-OS, along with a custom Central Processing Unit (CPU), the GTU-C312, upon which this OS will run. By simulating both the hardware and a significant portion of the system software, this endeavor provides a practical, in-depth exploration of fundamental OS concepts.

1.1. Background

Operating systems are complex software systems responsible for managing a computer's hardware and software resources, providing a platform for applications to run. As extensively detailed in seminal works such as Andrew S. Tanenbaum's "Modern Operating Systems," the core functions of an OS include process (or thread) management, memory management, file system management, I/O device management, and providing a user interface. These systems abstract the intricate details of the hardware, offering cleaner, more powerful, and more user-friendly interfaces to both programmers and end-users. Key concepts like concurrency, scheduling, synchronization, and inter-process communication are central to OS design, enabling efficient and fair utilization of system resources.

In the context of this semester project, we delve into these foundational principles by simulating a custom-designed CPU, the GTU-C312, and developing a bespoke operating system, GTU-OS, to run on it. The GTU-C312 features a unique, memory-mapped register architecture and a limited instruction set, presenting specific challenges and learning opportunities. The GTU-OS, written entirely in the GTU-C312's assembly language, must manage multiple threads of execution, implement a scheduling algorithm, handle system calls for basic I/O and thread control, and manage

thread states and context switches—all within the constraints of this hypothetical hardware. This hands-on approach allows for a deeper appreciation of the low-level mechanisms that underpin the sophisticated operating systems we use daily, echoing the complexities Tanenbaum describes in managing processes and their lifecycles.

1.2. Literature Review

The development of GTU-OS draws upon established operating system design principles and algorithms. Several key concepts from OS literature are directly implemented or simulated:

- **Thread Management and Thread Table**: Threads are the basic unit of CPU utilization in GTU-OS. To manage these, a **Thread Table** is implemented as a central data structure. Each entry in this table, often referred to as a **Thread Control Block** in literature, stores vital information about a thread, including its **ID**, **current state** (e.g., Ready, Running, Blocked, Halted), saved **Program Counter** (PC), saved **Stack Pointer** (SP), and accounting information such as **instruction usage** and **block duration**. The OS initializes this table at startup, creating entries for a predefined number of threads.
- **The OS as a Thread (Thread 0)**: In many operating system designs, and as implemented in GTU-OS, the OS kernel itself can be conceptualized or partially operate as a special, privileged thread. This "OS thread" is responsible for initialization, scheduling, handling system calls, and managing other threads. It has its own entry in the **Thread Table** to track its state and resource usage.
- **Scheduler and Round-Robin Algorithm**: The scheduler is the OS component responsible for deciding which thread should run next. GTU-OS implements a cooperative, round-robin scheduling algorithm. In this non-preemptive scheme, threads run until they voluntarily yield control (via a SYSCALL YIELD), terminate (SYSCALL HLT), or block on a system call (like SYSCALL PRN). The round-robin

algorithm ensures fairness by cycling through the list of ready threads, giving each an opportunity to run. The scheduler in GTU-OS maintains a pointer to the next thread to consider, advancing it after each selection.

- **Context Switching**: When the scheduler decides to run a different thread, a context switch must occur. This is the process of saving the state of the currently running thread and loading the saved state of the new thread that is about to run. The "context" includes the **Program Counter**, **Stack Pointer**, and potentially other CPU state (though in GTU-C312, general-purpose registers are memory-mapped and their saving/restoring is implicit if they are part of the thread's dedicated memory). The GTU-OS context switch routine carefully saves these to the outgoing thread's entry in the Thread Table and loads them from the incoming thread's entry.
- **System Calls**: System calls provide the interface between user threads and the OS kernel. When a thread needs to perform a privileged operation, it executes a SYSCALL instruction. This causes a trap to the OS, switching the CPU to kernel mode and transferring control to a specific OS handler routine. GTU-OS implements handlers for SYSCALL PRN, SYSCALL HLT, and SYSCALL YIELD.
- **Cooperative Multitasking**: Unlike preemptive multitasking where the OS can interrupt a thread at any time, GTU-OS uses cooperative multitasking. Threads are expected to "cooperate" by yielding control periodically. This simplifies the OS design as it doesn't need to handle complex preemption logic but relies on well-behaved threads.
- **Thread States**: Threads in GTU-OS transition between several states: **READY** (waiting for CPU), **RUNNING** (currently executing), **BLOCKED** (waiting for an event, e.g., I/O completion for SYSCALL PRN), and **HALTED** (terminated). The OS manages these state transitions based on thread actions and scheduler decisions.

1.3. Objectives

This semester project, as outlined in the official project description, aims to achieve a comprehensive understanding of operating system principles and low-level CPU architecture through hands-on implementation. The primary objectives are:

- **CPU and ISA Design Understanding**: To simulate a simple hypothetical CPU (GTU-C312) with a custom, small instruction set. This involves understanding memory-mapped registers, instruction formats, and the logic behind instruction execution (fetch-decode-execute cycle).
- **Operating System Implementation**: To write a simple cooperative operating system (GTU-OS) in the assembly-like language of the simulated GTU-C312 CPU. This OS must include:
    - **Thread Management**: Creating and managing a thread table, tracking thread states (Ready, Running, Blocked, Halted), and handling thread context switching.
    - **Cooperative Multitasking**: Implementing a scheduling algorithm (specifically round-robin) and managing thread execution via SYSCALL YIELD.
    - **System Call Handling**: Implementing handlers for system calls such as SYSCALL PRN (print), SYSCALL HLT (thread halt), and SYSCALL YIELD.
- **Systems Simulation**: To build a comprehensive CPU simulator in Python for the GTU-C312. This simulator is responsible for:
    - Accurately executing the GTU-C312 instruction set.
    - Managing simulated memory and enforcing access controls.
    - Loading programs written in GTU-C312 assembly.
    - Providing specified debug modes:
        - **Debug Mode 0**: Runs the program and prints a dump of all non-zero memory locations to the standard error stream after the CPU halts.

    - **Debug Mode 1**: Prints the contents of all non-zero memory locations to standard error after each CPU instruction execution.
    - **Debug Mode 2**: Similar to Debug Mode 1, but also waits for a keypress after each instruction, allowing for step-by-step execution.
    - **Debug Mode 3**: The simulator prints a formatted dump of the OS's Thread Table to standard error. This occurs after the execution of a USER instruction and after the CPU-level processing of any SYSCALL instruction. Additionally, in this mode, the final state of the Thread Table is printed when the simulation concludes.
- **Algorithmic Problem Solving**: To implement low-level algorithms (sorting, searching) and custom logic using the GTU-C312 instruction set as separate threads running under the GTU-OS. This involves writing at least three distinct threads: one for sorting, one for linear search, and one custom-designed thread.

2. METHODOLOGY

This section describes how the project was implemented, including the custom data structures, the spell checking algorithm, and the performance-related design decisions.

2.1. Simulation Environment and Hypothetical CPU

The foundation of this project is the software simulation of the GTU-C312 Central Processing Unit. This section outlines the simulation environment's architecture, the specific design of the GTU-C312 CPU, its instruction set, and its operational cycle as defined by the project requirements.

2.1.1. Overview of the Simulation

The simulation operates as a discrete-time system where each "tick" corresponds to the execution of a single GTU-C312 instruction. The simulation environment is modular, consisting of:

- **Main Simulator (main.py)**: Orchestrates the simulation, parses command-line arguments (program filename, debug level), loads the program, and manages the main execution loop.
- **CPU Module (gtu_cpu.py)**: Implements the GTU-C312 CPU logic. This includes managing the CPU's internal state (registers, execution mode), fetching instructions from memory, decoding them, and executing the corresponding operations.
- **Memory Module (memory.py)**: Simulates the CPU's main memory. It provides mechanisms for reading and writing integer values to specific memory addresses and enforces memory access restrictions based on the CPU's current execution mode.
- **Instruction Module (instruction.py)**: Defines the Instruction class, a data structure used to represent a parsed assembly instruction, holding its opcode and operands.
- **Program Loader Module (loader.py)**: Responsible for reading and parsing .gtu312 program files. It distinguishes between the "DATA SECTION" for initial memory values and the "INSTRUCTION SECTION" for program code, converting them into a format usable by the CPU and memory modules.
- **Utilities Module (utils.py)**: Contains shared enumerations, primarily CPUMode (defining KERNEL and USER modes) and ThreadState (for OS-level thread management, though ThreadState itself is not directly used by the gtu_cpu.py module).

The simulation begins with the **ProgramLoader** parsing the specified **.gtu312** file. The CPU module is then initialized with this program data and instructions. The main simulation loop in **main.py** repeatedly calls the **CPU's step()** method, which executes one instruction per call, until the CPU signals a halt condition or a predefined maximum cycle count is reached.

2.1.2. The GTU-C312 Hypothetical CPU Architecture

The **GTU-C312** is a custom-designed hypothetical CPU. It follows a Von Neumann architecture, where instructions and data reside in the same addressable memory space. Its architecture is characterized by its memory-mapped registers, memory organization, specific instruction set, dual execution modes, and system call mechanism.

1. **Registers (Memory-Mapped):**

   As specified, the GTU-C312 CPU does not possess dedicated hardware registers in the traditional sense. Instead, critical CPU state information is stored in reserved, special memory locations. These are:

   - **Memory Location 0 (Program Counter - PC)**: Stores the memory address of the next instruction to be fetched and executed. This is represented by CPU.PC_ADDR in the simulator.
   - **Memory Location 1 (Stack Pointer - SP)**: Stores the memory address of the current top of the stack. The stack grows downwards in memory. This is CPU.SP_ADDR.
   - **Memory Location 2 (System Call Result)**: Used to return simple status information or results from system call operations. For instance, SYSCALL PRN uses this to indicate the blocking duration, and SYSCALL HLT or SYSCALL YIELD could use it for status codes if the OS were to interpret them. This is CPU.SYSCALL_RESULT_ADDR.
   - **Memory Location 3 (Number of Instructions Executed)**: A counter that is incremented after each instruction execution, providing a measure of CPU work. This is CPU.INST_COUNT_ADDR
   - **Memory Location 4**: Used by the CPU to store the return Program Counter from a user thread when a SYSCALL instruction is executed. This is referred to as

CPU.SYSCALL_SAVE_PC_ADDR in the simulator. Memory Locations 5-20: Reserved for other uses..

- **Memory Locations 5-20**: Reserved for other uses, potentially by the Operating System for global variables or further CPU state if the architecture were extended.

These memory-mapped registers are accessed by the CPU's internal logic primarily through the MemoryManager's get_value_unchecked and set_value_unchecked methods to bypass mode restrictions for fundamental CPU operations.

2. **Memory Model:**

The CPU interacts with a single, linear address space managed by the MemoryManager class.

- **Size**: The total memory size is configurable at simulation startup (defaulting to 20,000 addressable integer units in the provided simulator code).
- **Addressing**: Each memory location can store a signed integer value. Addresses range from 0 to size-1.
- **Access Control and Protection**: The MemoryManager enforces memory access restrictions based on the CPU's current operating mode:
- **Kernel Mode**: The CPU has unrestricted read/write access to the entire memory space. The CPU starts in Kernel mode.
- **User Mode** : User mode programs can only access memory locations from address 1000 (MemoryManager.USER_MEMORY_START) upwards. Any attempt by a user mode instruction to access an address below 1000 (e.g., to modify memory-mapped registers or OS data) results in a MemoryAccessError, which, as per project specification, should lead to the offending thread's shutdown (handled by the simulator halting or the OS terminating the thread).

- **Stack**: A runtime stack is implemented within this memory space. It is used for subroutine calls (CALL, RET) and explicit data manipulation (PUSH, POP). The stack grows downwards from higher memory addresses towards lower ones, managed by the SP. The simulator includes checks for stack overflow (SP becoming too small) and stack underflow (SP becoming too large).

3. **Instruction Set Architecture (ISA):**

   The GTU-C312 features a small and non-typical instruction set, as defined in the project specification. Instructions are processed by the CPU module. Operands can be:

   - **B**: An immediate signed integer value.
   - **A, A1, A2**: Memory addresses.
   - **C**: An instruction number (target address for jumps/calls).

   The instruction set is as follows:

Instruction	**Operands**	**Explanation**
SET	B A	**Direct Set:** Writes the immediate value B to memory location A. (M[A] = B)
CPY	A1 A2	**Direct Copy**: Copies the integer value from memory location A1 to memory location A2. (M[A2] = M[A1])
CPYI	A1 A2	**Indirect Copy**: Reads an address ptr from memory location A1. Copies the value from M[ptr] to M[A2]. (M[A2] = M[M[A1]])
CPYI2	A1 A2	**Indirect Copy 2**: Reads source address src_ptr from memory location A1. Reads destination address dest_ptr from memory location A2. Copies the value from memory location src_ptr to memory location dest_ptr. (Effectively, M[M[A2]] = M[M[A1]])
ADD	A B	**Add**: Adds the immediate value B to the integer value at memory location A, storing the result in A. (M[A] = M[A] + B)
ADDI	A1 A2	**Indirect Add**: Adds the content of memory address A1

		to the content of memory address A2, storing the result in memory location A2. (M[A2] = M[A2] + M[A1])
SUBI	A1 A2	**Indirect Subtraction**: Subtracts the content of memory address A2 from the content of memory address A1, storing the result in A2. (M[A2] = M[A1] - M[A2])
JIF	A C	**Jump If**: If the value at memory location A is less than or equal to 0, sets the PC to C. Otherwise, PC increments normally.
PUSH	A	**Push** memory A onto the stack. The stack pointer (SP) is decremented before the write.
POP	A	**Pop** value from stack into memory A. The stack pointer (SP) is incremented after the read.
CALL	C	**Call Subroutine**: Pushes the return address (current PC + 1) onto the stack, then sets PC to C.
RET	-	**Return**: Pops the return address from the stack and sets the PC to this address.
HLT	-	**Halt**: Stops all CPU execution. The simulator's halted flag is set.
USER	A	**Switch to User Mode**: Reads a starting PC value for user mode from memory location A. Switches CPU mode to USER and sets PC to this new value.
SYSCALL PRN	A	**System Call Print:** Reads an address, ptr_val, from memory location A. Then, prints the integer content of memory location ptr_val (i.e., M[M[A]]) to the console, followed by a newline. This system call is specified to block the calling thread for 100 instruction executions, a behavior managed by the OS which sets an appropriate block duration for the thread.
SYSCAL HLT	-	**System Call Halt Thread**: Signals the Operating System to shut down the currently executing thread.
SYSCALL YIELD	-	**System Call Yield**: Signals the Operating System that the current thread voluntarily relinquishes the CPU, allowing the OS to schedule other threads.

4. Instruction Set Architecture (ISA):

The GTU-C312 CPU operates in one of two modes:

- **CPUMode.KERNEL**: The privileged mode. The CPU starts in this mode. All memory is accessible, and all instructions can be executed. Typically, the OS code runs in Kernel mode.
- **CPUMode.USER**: The unprivileged mode, intended for user application threads. Memory access is restricted (see Memory Model).
- Mode switching occurs under specific conditions:
    - **Kernel to User**: Execution of the USER A instruction.
    - **User to Kernel**: Automatically upon execution of any SYSCALL instruction. This allows user programs to request privileged operations from the OS. The CPU returns to User mode when the OS decides to.

5. **Syscall:**

   When any SYSCALL instruction is executed by a thread in User mode, the CPU automatically performs the following actions:

   - Switches its operating mode from CPUMode.USER to CPUMode.KERNEL.
   - Saves the Program Counter of the instruction immediately following the SYSCALL instruction (i.e., current PC + 1 of the user thread) into memory location 4 (CPU.SYSCALL_SAVE_PC_ADDR).
   - Transfers control by setting its Program Counter to a specific, fixed entry point within the Operating System's code, based on the type of system call:
       - **SYSCALL YIELD**: the CPU jumps to OS instruction address 500.
       - **SYSCALL HLT**: the CPU jumps to OS instruction address 550.
       - **SYSCALL PRN A**: after the Python CPU simulation environment handles the actual printing to the console (as per its implementation of printing M[M[A]]), the CPU jumps to OS instruction address 600. This allows

the OS to manage thread blocking and subsequent scheduling.

-

6. **CPU Execution Cycle (Fetch-Decode-Execute):**

   The CPU's operation is driven by its step() method, which simulates one fetch-decode-execute cycle:

   1. **Halt Check**: If the CPU is already halted, no action is taken.
   2. **Fetch**: The instruction at the memory address specified by the PC is fetched. PC validity is checked.
   3. **Decode**: The fetched instruction's opcode and operands are identified.
   4. **Execute**: The operation corresponding to the opcode is performed. This may involve:
      a. Reading from or writing to memory (respecting current CPU mode).
      b. Performing arithmetic operations.
      c. Modifying the PC (for jumps, calls, returns).
      d. Changing the CPU mode.
      e. Interacting with the stack.
      f. Setting the halted flag.
   5. **PC Update**: If the executed instruction did not modify the PC directly (e.g., JIF, CALL, RET, USER), the PC is incremented to point to the next sequential instruction.
   6. **Instruction Count Update**: The master instruction counter at INST_COUNT_ADDR (memory location 3) is incremented. This cycle repeats until the HLT instruction is executed or a fatal, unrecoverable error occurs (e.g., invalid PC, memory access violation leading to simulator halt).

2.1.3. Program Loading and Format

Programs for the GTU-C312 are written in .gtu312 files. As specified, these files have two main sections:

- **BEGIN DATA SECTION ... END DATA SECTION**: This section contains pairs of **<address> <value>,** specifying initial integer values to be loaded into memory before execution begins. This is used for setting up initial register values (like PC and SP if they are not hardcoded to start at 0 and top-of-stack respectively by the CPU logic itself), global variables, and thread data.
- **BEGIN INSTRUCTION SECTION ... END INSTRUCTION SECTION**: This section contains lines of the format **<instruction_number> <OPCODE> [OPERANDS...]**. These are parsed into Instruction objects and stored in a list accessible by the CPU, indexed by their instruction number. **The ProgramLoader** module handles parsing these sections and populating the **MemoryManager** and the CPU's instruction list accordingly.

Programs for the GTU-C312 are written in .gtu312 files. As specified, these files have two main sections

Sample Code:

```
# Sample program - adds numbers from 1 to 10
# Result should be 55 (1+2+3+4+5+6+7+8+9+10)
# This matches the exact format from the PDF

Begin Data Section
0 0     # program counter
1 0     # stack pointer
2 0     # syscall result
3 0     # instruction count
4 0     # reserved
5 0     # reserved
6 0     # reserved
7 0     # reserved
8 0     # reserved
9 0     # reserved
10 0    # reserved
11 0    # reserved
12 0    # reserved
13 0    # reserved
14 0    # reserved
15 0    # reserved
```

```
16 0    # reserved
17 0    # reserved
18 0    # reserved
19 0    # reserved
20 0    # reserved
21 0
22 0
23 0
24 0
25 0
26 0
27 0
28 0
29 0
30 0
31 0
32 0
33 0
34 0
35 0
36 0
37 0
38 0
39 0
40 0
41 0
42 0
43 0
44 0
45 0
46 0
47 0
48 0
49 0
50 0    # will store i (counter)
51 0    # will store sum (result)
52 0
53 0
54 0
55 0
56 0
57 0
58 0
59 0
60 0
61 0
62 0
63 0
64 0
```

65 0
66 0
67 0
68 0
69 0
70 0
71 0
72 0
73 0
74 0
75 0
76 0
77 0
78 0
79 0
80 0
81 0
82 0
83 0
84 0
85 0
86 0
87 0
88 0
89 0
90 0
91 0
92 0
93 0
94 0
95 0
96 0
97 0
98 0
99 0
100 0
101 0
102 0
103 0
104 0
105 0
106 0
107 0
108 0
109 0
110 0
111 0
112 0
113 0

114 0
115 0
116 0
117 0
118 0
119 0
120 0
121 0
122 0
123 0
124 0
125 0
126 0
127 0
128 0
129 0
130 0
131 0
132 0
133 0
134 0
135 0
136 0
137 0
138 0
139 0
140 0
141 0
142 0
143 0
144 0
145 0
146 0
147 0
148 0
149 0
150 0
151 0
152 0
153 0
154 0
155 0
156 0
157 0
158 0
159 0
160 0
161 0
162 0

163 0
164 0
165 0
166 0
167 0
168 0
169 0
170 0
171 0
172 0
173 0
174 0
175 0
176 0
177 0
178 0
179 0
180 0
181 0
182 0
183 0
184 0
185 0
186 0
187 0
188 0
189 0
190 0
191 0
192 0
193 0
194 0
195 0
196 0
197 0
198 0
199 0
200 0
201 0
202 0
203 0
204 0
205 0
206 0
207 0
208 0
209 0
210 0
211 0

```
212 0
213 0
214 0
215 0
216 0
217 0
218 0
219 0
220 0
221 0
222 0
223 0
224 0
225 0
226 0
227 0
228 0
229 0
230 0
231 0
232 0
233 0
234 0
235 0
236 0
237 0
238 0
239 0
240 0
241 0
242 0
243 0
244 0
245 0
246 0
247 0
248 0
249 0
250 0
251 0
252 0
253 0
254 0
255 0
End Data Section

Begin Instruction Section
0 SET 10 50     # i = 10
1 SET 0 51      # sum = 0
```

```
2 ADDI 50 51    # sum = sum + i
3 ADD 50 -1     # i = i - 1
4 JIF 50 7      # Go to 7 if i <= 0
5 SYSCALL PRN 51 # print the final sum
                # This will be ignored
6 SET 2 0       # Go to 2 (set PC to 2)
7 HLT           # end of program
End Instruction Section
```

2.2. Operating System (GTU-OS)

The Operating System (GTU-OS) for the GTU-C312 CPU is implemented entirely in the GTU-C312 assembly language, adhering to the instruction set defined earlier. Its primary responsibilities include thread management, cooperative multitasking via a round-robin scheduling algorithm, and handling system calls initiated by threads. The OS itself operates as a special thread (Thread 0) and manages up to 10 additional user threads (Threads 1-10).

2.2.1. Core Data Structures and Memory Layout

The GTU-OS utilizes specific memory regions for its operation and for managing threads:

- **OS Variables (Memory Addresses 21-99)**: This region holds global OS state information, including:
    - **current_thread_ID (mem[21])**: The ID of the thread currently considered "running" by the OS.
    - **num_active_threads (mem[22])**: A counter for threads not in the HALTED state.
    - **next_thread_to_check (mem[27])**: Used by the scheduler to remember the starting point for the next round-robin search, ensuring fairness.

    - **thread_entry_size (mem[32])**: Fixed size of each thread's entry in the Thread Table (80 bytes).
    - **thread_base_offset (mem[33])**: Starting memory address of the Thread Table (100).
    - **last_thread_execution_start (mem[34])**: Stores the CPU's total instruction count (from mem[3]) when a user thread begins its execution quantum.
    - **instruction_count_of_last_executed_thread** (mem[35]): Stores the CPU's total instruction count (from mem[3]) when a user thread finishes its quantum (e.g., makes a syscall). These two (34 & 35) are used to calculate the duration of the last quantum.
- **Thread Table (TT) (Memory Addresses 100-979)**: This is the central data structure for thread management. It contains entries for the OS (Thread 0) and up to 10 user threads. Each 80-byte entry stores:
    - **Offset 0:** Thread ID (0 for OS, 1-10 for user threads).
    - **Offset 1:** State:
        - **1 (READY)**: Thread is ready to run but waiting for the CPU.
        - **2 (RUNNING)**: Thread is currently executing (conceptually, from OS perspective).
        - **3 (BLOCKED)**: Thread is waiting for an event (e.g., SYSCALL PRN completion).
        - **4 (HALTED)**: Thread has finished execution or been terminated.
    - **Offset 2**: Saved Program Counter (PC) for the thread.
    - **Offset 3**: Saved Stack Pointer (SP) for the thread.
    - **Offset 4**: first_execution_time: CPU instruction count when the thread first ran (or -1 if never run).
    - **Offset 5**: last_start_time: Holds the cumulative instruction usage by the thread before its current quantum starts. This is effectively used to sum up previous quanta.

- **Offset 6**: total_executed_instructions: Total number of CPU instructions consumed by this thread throughout its lifetime. For the OS thread, this tracks total OS instruction usage.
- **Offset 7**: IC_of_the_last_executed_instruction: For BLOCKED threads, this stores the CPU's total instruction count at the moment the thread became blocked.
- **Offset 8**: block_duration: For BLOCKED threads (like after SYSCALL PRN), this stores the number of CPU instruction cycles the thread must wait.

2.2.2. OS Initialization, Thread Table Setup, and Main Loop

The GTU-OS begins execution when the CPU's Program Counter is initialized to the OS entry point (instruction 21 in the OS code). The initialization process involves two main phases: setting up the Thread Table for all potential threads (including the OS itself as Thread 0 and user threads 1-10) and then entering the main scheduling loop.

1. **Thread Table Initialization (OS Instructions 21-68)**:

   Upon starting, the OS systematically initializes the Thread Table located in memory from address 100 onwards. For each thread from ID 1 up to the maximum number of threads (10, defined in mem[31] as 11 to include thread 0, but the loop effectively processes 1 through 10 for user threads after initializing thread 0 implicitly or as part of a general structure):

   - **Calculate Thread Table Entry Address**: The base address for each thread's 80-byte entry in the Thread Table is calculated as: thread_base_offset (mem[33], which is 100) + (thread_id * thread_entry_size (mem[32], which is 80)). The OS uses temporary memory locations (e.g., mem[10] for the current thread_id being processed, mem[13-15] for

calculations) to perform this multiplication and addition.

- **Populate Thread Table Entry**: For each user thread (ID 1-10), the OS populates its 80-byte entry as follows:
    - **Offset 0 (Thread ID)**: The current thread_id is stored.
    - **Offset 1 (State)**: Initialized to 1 (READY).
    - **Offset 2 (Saved PC)**: The starting Program Counter for the thread is calculated as thread_id * 1000. So, Thread 1 starts at PC 1000, Thread 2 at PC 2000, and so on.
    - **Offset 3 (Saved SP)**: The initial Stack Pointer for the thread is set to (calculated_PC_for_thread) + 999. This places the stack at the end of the thread's 1000-byte allocated memory block (e.g., SP for Thread 1 is 1999).
    - **Offset 4 (first_execution_time)**: Initialized to 0.
    - **Offset 5 (last_quantum_start_IC**): Initialized to 0.
    - **Offset 6 (total_executed_instructions)**: Initialized to 0.
    - **Offset 7 (IC_of_the_last_executed_instruction)**: Initialized to 0.
    - **Offset 8 (block_duration)**: Initialized to 0.

2. **Enter Main Scheduler (OS Instruction 68 onwards):**
    a. Once the thread table initialization loop is complete (after processing up to the maximum thread ID), the OS sets its internal scheduler state flag (mem[23]) to 1, indicating that the system is fully initialized and normal scheduling can begin (OS instruction 68 SET 1 23).

b. It then unconditionally jumps to the main scheduler routine, which starts at OS instruction 100 (OS instruction 69 SET 100 0).

3. **Scheduler Main Loop (OS Instruction 100 onwards)**:

   The OS enters its primary scheduling loop:

   - **Check for Active Threads**: It first checks if any active threads remain by inspecting num_active_threads (mem[22]). If mem[22] is zero or less, the OS prepares for system shutdown. It proceeds to update its own statistics in Thread 0's table entry (specifically, calculating its final cumulative user thread execution tally in TT0[Offset 5] and its own net execution time in TT0[Offset 6]) before executing the HLT instruction (at OS address 125) to stop the CPU.
   - **Find Next Thread**: If active threads exist, the OS calls the "Find Next Thread" routine (starting at OS instruction 250) to select a user thread to run based on the round-robin policy.
   - **Context Switch to User Thread**: If a suitable, READY user thread is found (its ID is returned in mem[16]), the OS prepares for a context switch. It sets the new state for the current thread (which is the OS itself at this point) to READY (mem[17]=1) and then jumps to the "Context Switch" routine (starting at OS instruction 350). This routine will save the OS's context and load the selected user thread's context.
   - **Loop if No Ready Thread**: If the "Find Next Thread" routine does not find any user thread in the READY state, mem[16] will be 0. The scheduler then loops back (OS instruction 106 SET 100 0), effectively idling or re-evaluating until a thread becomes ready (e.g., unblocked).

2.2.3. Scheduling Algorithm (Round-Robin with Block Handling)

The Find Next Thread routine (starting at instruction 250) implements a round-robin scheduling policy:

1. **Search Initiation**: It starts searching from the thread ID stored in next_thread_to_check (mem[27]), wrapping around to thread 1 after checking the highest numbered thread. It explicitly skips Thread 0 (the OS) in this user thread selection process.
2. **State Evaluation**: For each thread, it calculates the address of its entry in the Thread Table and reads its state:
    a. **READY (State 1)**: If a thread is READY, it is selected for execution. The ID of this chosen thread is stored in mem[16]. next_thread_to_check is updated to the thread after the selected one for the next scheduling cycle. The routine then returns to the main scheduler.
    b. **BLOCKED (State 3)**: If a thread is BLOCKED, the OS checks if its blocking condition has expired. This is done by:
        i. Reading IC_of_the_last_executed_instruction (TT offset 7 - when it was blocked) and block_duration (TT offset 8).
        ii. Comparing (IC_when_blocked + block_duration) with the current CPU total instruction count (mem[3]).
        iii. If the block duration has passed, the thread's state is changed to READY, its block_duration is reset to 0, and it becomes a candidate for selection in the current scheduling pass. This mechanism, using a time-based check for resource release (CPU availability after a PRN call), is reminiscent of ensuring processes don't wait indefinitely, a principle discussed in operating systems literature like Tanenbaum's "Operating

Systems," where managing resource wait times is crucial, though here it's a fixed instruction count rather than a dynamic working set.

c. **HALTED (State 4) or RUNNING (State 2)**: These threads are skipped.

3. **No Ready Thread**: If the loop completes without finding a READY or newly unblocked thread, mem[16] remains 0 (or is set to 0), indicating no thread is ready to run.

2.2.4. Context Switching

Context switching is the core mechanism for transferring CPU control. It occurs when the scheduler has selected a new user thread to run, or when a user thread makes a system call, thereby returning control to the OS.

- **Input Parameters**: The context switch routine expects the ID of the new thread to run in mem[16] and the desired new state for the outgoing thread in mem[17].
- **Saving Outgoing Thread's Context**:
    - The OS identifies the outgoing thread (from current_thread_ID, mem[21]).
    - It calculates the outgoing thread's TT entry.
    - **For a User Thread (ID > 0)**:
        - The new state (from mem[17]) is saved to its TT entry.
        - The PC (from SYSCALL_SAVE_PC_ADDR, mem[4] – where the CPU saved it during syscall) is saved to its TT PC field.
        - The current SP (from CPU mem[1]) is saved to its TT SP field.
        - **Instruction Counts:**
            - The number of instructions executed in the just-completed quantum is mem[35] (IC_when_syscall_occurred) - mem[34] (IC_when_thread_started_quantum).

        - The number of instructions executed in the just-completed quantum is calculated as instruction_count_of_last_executed_thread (mem[35]) minus last_thread_execution_start (mem[34]). This quantum length is then added to the thread's previously accumulated total_executed_instructions (retrieved from its TT Offset 6), and the new sum is stored back into TT Offset 6.
        - mem[35] is saved as IC_of_the_last_executed_instruction (TT offset 7).
    - If the new state is BLOCKED (e.g., after SYSCALL PRN), the block_duration (TT offset 8) is set to 100 (instruction 481).
- **For the OS Thread (ID = 0)**:

  When its context is saved (typically after handling a user thread's syscall and before scheduling another user thread, or before the final system halt):

  1. Its state is updated in TT0[Offset 1].
  2. A specific procedure updates its instruction accounting in its Thread Table entry (TT0):
     a. The duration of the user thread's quantum that just finished is added to the value currently stored in TT0[Offset 5]. Thus, TT0[Offset 5] cumulatively tracks the total instruction cycles spent by all user threads executed so far.
     b. The OS's own total_executed_instructions is then updated. It is calculated as the current total CPU instruction count minus the new cumulative user thread execution time now stored in TT0[Offset 5]. This difference

represents the net instruction cycles consumed by the OS itself up to that point.

- **Loading Incoming Thread's Context:**
  - The OS sets current_thread_ID (mem[21]) to the new thread's ID (from mem[16]).
  - It calculates the incoming thread's TT entry.
  - **For a User Thread (ID > 0)**:
    - Its state in TT is set to RUNNING (2).
    - Its saved PC from TT is loaded into mem[18] (the operand for the USER instruction).
    - Its saved SP from TT is loaded into the CPU's SP (mem[1]).
    - The current CPU instruction count (mem[3]) is stored in last_thread_execution_start (mem[34]) to mark the beginning of its new quantum.
    - The OS then executes USER mem[18], transferring control to the user thread in user mode.
  - If the Incoming Thread is OS (ID = 0): This typically happens after a syscall. The OS's SP is already active. Its state is set to RUNNING. The OS then jumps back to the main scheduler loop (instruction 100).

2.2.5. System Call Handling

When a user thread executes a SYSCALL instruction, the CPU automatically:

1. Switches to KERNEL mode.
2. Saves the user thread's next PC (current_PC + 1) into SYSCALL_SAVE_PC_ADDR (mem[4]).
3. Jumps to a predefined OS handler address based on the syscall type.

The OS provides handlers for:

- **SYSCALL YIELD (Handler at 500):**

    - The OS records the current CPU IC (mem[3]) into instruction_count_of_last_executed_thread (mem[35]).
    - It sets parameters for context switch: new thread is OS (mem[16]=0), current (yielding) thread's new state is READY (mem[17]=1).
    - Jumps to Context Switch (350).
- **SYSCALL HLT (Handler at 550):**
    - Records IC into mem[35].
    - Sets parameters: new thread is OS (mem[16]=0), current (halting) thread's new state is HALTED (mem[17]=4).
    - Decrements num_active_threads (mem[22]).
    - Jumps to Context Switch (350).
- **SYSCALL PRN A (Handler at 600 - CPU performs print, OS handles blocking):**
    - Records IC into mem[35].
    - Sets parameters: new thread is OS (mem[16]=0), current (printing) thread's new state is BLOCKED (mem[17]=3).
    - Jumps to Context Switch (350). The context switch routine will then set this thread's block_duration in its TT entry to 100.

2.2.6. Overall Operational Flow

The system operates in a cycle:

1. OS initializes and enters the scheduler.
2. Scheduler selects a ready user thread.
3. Context switch occurs: OS state saved, user thread state loaded, CPU switches to USER mode, user thread PC is loaded.
4. User thread executes its instructions.
5. User thread performs a SYSCALL (YIELD, HLT, or PRN).
6. CPU switches to KERNEL mode, saves user PC, jumps to OS syscall handler.
7. OS syscall handler prepares for context switch back to OS.

8. Context switch occurs: User thread state saved (e.g., as READY or BLOCKED), OS state loaded.
9. OS (now running) jumps back to the scheduler to pick the next thread.
10. This cycle continues until all user threads are HALTED, at which point the OS will execute the HLT instruction to stop the CPU.

2.2.7. User Thread Implementation

To validate the GTU-OS and demonstrate practical application of the GTU-C312 instruction set, several user threads were developed. Each thread was implemented entirely in GTU-C312 assembly, adhering to the OS's memory allocation scheme (1000-byte blocks starting at M[1000], M[2000], etc.) and utilizing the system calls provided by GTU-OS for interaction and control.

- **Thread 1: Bubble Sort (Instructions M[1000-1051], Data M[1001-1025]):**
    - **Objective**: To sort an array of 10 predefined integers in ascending order.
    - **Implementation Details**: The thread initializes loop counters (i at M[1012], j at M[1013]) for the nested loops characteristic of bubble sort. Array elements are stored starting at M[1002]. The core logic involves:
        - Iteratively comparing adjacent array elements (arr[j] and arr[j+1]). Memory addresses of these elements are calculated dynamically by adding the loop counter j to the base address of the array (M[1002]).
        - CPYI is used to fetch values from calculated array addresses, and SUBI is used for comparison.
        - If elements are out of order (arr[j] > arr[j+1]), a swap is performed. This involves using temporary memory locations (e.g., M[1014], M[1015]) to hold values and CPYI2 to write the swapped values back to their

respective array positions using pointers to these temporary locations and calculated array element addresses.
        - JIF instructions control the flow of the inner and outer loops based on counter comparisons against array size limits (N at M[1001]).
- **Thread 2: Linear Search (Instructions M[2000-2023], Data M[2000-2030])**
    - **Objective**: To find the first occurrence of a specific KEY (M[2021]) within an array of 8 integers (array base address stored in M[2022], data from M[2023]).
    - **Implementation Details**:
        - A loop counter I (M[2000]) iterates from 0 to N-1 (N at M[2020]). The result index is stored in M[2001] (initialized to -1).
        - In each iteration, the address of the current array element array[I] is calculated by adding I to the array's base address value (M[M[2022]]).
        - CPYI fetches array[I], and SUBI compares it with the KEY.
        - If a match is found (KEY - array[I] == 0), the current value of I is copied to M[2001], and the loop terminates by jumping to the print routine.
        - JIF instructions manage loop continuation and termination.
- **Thread 3: Iterative Fibonacci Sequence (Instructions M[3000-3030], Data M[3001-3012])**
    - **Objective**: To calculate and print the first N (M[3001], e.g., 10) Fibonacci numbers.
    - **Implementation Details**:
        - The thread initializes variables for fib(n-2) (M[3003]=0) and fib(n-1) (M[3004]=1). A counter (M[3002]) tracks the number of Fibonacci terms generated.

- It first prints the initial two values (0 and 1) if N is sufficient.
- The main loop then iteratively calculates the next Fibonacci number (current_fib at M[3005]) by adding fib(n-1) and fib(n-2) using ADDI.
- After printing current_fib, values are shifted: fib(n-2) becomes the old fib(n-1), and fib(n-1) becomes the current_fib.
- JIF controls the loop based on the counter against N.

3. Results and Discussion

This section presents the outcomes of simulating the GTU-C312 CPU and the GTU-OS with its accompanying user threads. The behavior of the system was observed using the four distinct debug modes provided by the simulator. Each debug mode offers unique insights into the system's operation, from high-level memory states to detailed thread management information.

1. **Simulation Setup**
    a. **Program File**: All simulations utilized operating_system.gtu312, which contains the GTU-OS, its data, the Thread Table structure, and all user threads with their respective code and initial data.
    b. **Simulator Parameters**: All simulations were run using the default parameters of the Python-based GTU-C312 simulator. Specifically, the memory size (--memsize) was set to the default of 20,000 integer locations, and the maximum simulation cycles (--maxcycles) was set to the default of 500,000. No non-default parameters were used for the generation of these results.
    c. **Threads Implemented**: The system includes the OS (Thread 0) and the following key user threads, each with data and

instructions placed in their OS-assigned memory blocks (e.g., Thread 1 in 1000-1999, etc.):

i. **Thread 1 (Bubble Sort)**: Sorts N integers (data from M[1001], instructions from M[1000]) and prints the sorted array.

ii. **Thread 2 (Linear Search)**: Searches for a KEY in N integers (data/variables from M[2000], array from M[2023], instructions from M[2000]) and prints the result index.

iii. **Thread 3 (Fibonacci Sequence)**: Calculates and prints N Fibonacci numbers (data from M[3001], instructions from M[3000]).

iv. **Threads 4-10 (Halt)**: Minimal threads executing SYSCALL HLT, demonstrating OS capacity for managing up to 10 user threads. Their respective data/instruction blocks start at M[4000], M[5000], etc.

2. **Debug Mode 0: Final System State**

Debug Mode 0 provides a snapshot of the entire system's non-zero memory contents after the CPU simulation has halted. This allows for an analysis of the final state of the OS, its managed threads, and the results of thread computations. The simulation was run with the operating_system.gtu312 program file.

Execution Overview:

The simulation proceeded with the GTU-OS initializing itself and the thread table. It then entered its round-robin scheduling loop, dispatching user threads (Bubble Sort, Linear Search, Fibonacci, and placeholder Halt threads). These threads executed their respective algorithms, making system calls for printing (SYSCALL PRN), yielding control (SYSCALL YIELD), or terminating (SYSCALL HLT). The OS handled these syscalls, managed context switches, and updated thread states. The simulation concluded when the OS executed its HLT instruction, typically after all user threads had terminated.

Output Screenshot:

```
emir@lenovo:~/Documents/github/OS$ python3 gtu_cpu_simulatator/main.py programs/operating_system.gtu312
Simulator initialized for 'programs/operating_system.gtu312'.
Debug: 0, MemSize: 20000, MaxCycles: 500000
Loading program: programs/operating_system.gtu312...
Program loaded into CPU successfully.

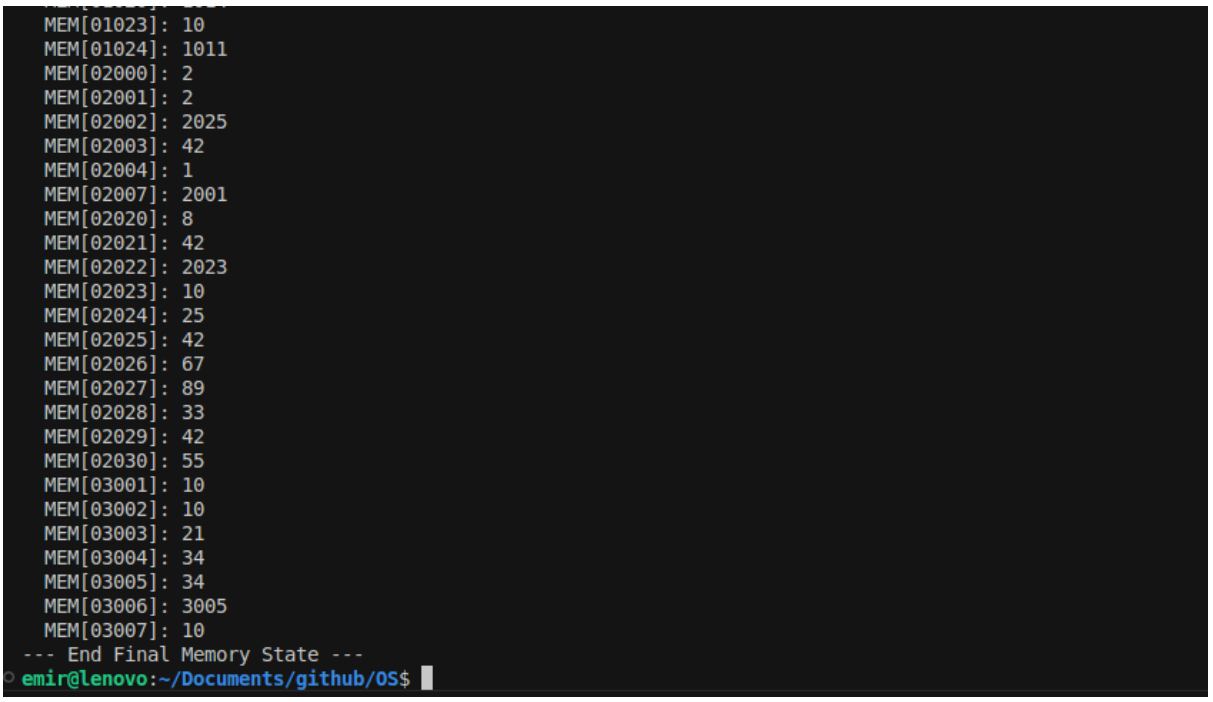
Starting CPU simulation (Debug Level: 0)...
2
0
11
1
12
23
1
34
2
45
3
56
5
67
8
78
13
89
21
90
34

================================================
CPU Simulation Finished
================================================
Status: HALTED
Total CPU cycles simulated: 13213
Total instructions executed: 13213
Final PC: 129
Final SP: 19999
CPU Mode: KERNEL

--- Final Memory State ---
PC: 129 SP: 19999 Mode: KERNEL TotalInst: 13213
  MEM[00000]: 129
  MEM[00001]: 19999
  MEM[00003]: 13213
  MEM[00004]: 3031
  MEM[00010]: 1396
  MEM[00011]: 11805
  MEM[00012]: 13112
  MEM[00013]: 13209
  MEM[00014]: 106
  MEM[00015]: 11
  MEM[00016]: 15
  MEM[00017]: 4
  MEM[00018]: 21
  MEM[00019]: 10000
  MEM[00023]: 1
```

```
MEM[00025]: 66
MEM[00027]: 4
MEM[00029]: 1
MEM[00030]: 4
MEM[00031]: 11
MEM[00032]: 80
MEM[00033]: 100
MEM[00034]: 13104
MEM[00035]: 13112
MEM[00101]: 4
MEM[00102]: 21
MEM[00103]: 19999
MEM[00105]: 1404
MEM[00106]: 11805
MEM[00180]: 1
MEM[00181]: 4
MEM[00182]: 1052
MEM[00183]: 1999
MEM[00184]: 929
MEM[00185]: 12867
MEM[00186]: 1242
MEM[00187]: 12872
MEM[00260]: 2
MEM[00261]: 4
MEM[00262]: 2024
MEM[00263]: 2999
MEM[00264]: 2269
MEM[00265]: 4576
MEM[00266]: 48
MEM[00267]: 4577
MEM[00340]: 3
MEM[00341]: 4
MEM[00342]: 3031
MEM[00343]: 3999
MEM[00344]: 2507
MEM[00345]: 13104
MEM[00346]: 107
MEM[00347]: 13112
MEM[00420]: 4
MEM[00421]: 4
MEM[00422]: 4001
MEM[00423]: 4999
MEM[00424]: 2714
MEM[00425]: 2714
MEM[00426]: 1
MEM[00427]: 2715
MEM[00500]: 5
MEM[00501]: 4
MEM[00502]: 5001
MEM[00503]: 5999
MEM[00504]: 2927
MEM[00505]: 2927
MEM[00506]: 1
MEM[00507]: 2928
MEM[00580]: 6
MEM[00581]: 4
```

```
MEM[00582]: 6001
MEM[00583]: 6999
MEM[00584]: 3152
MEM[00585]: 3152
MEM[00586]: 1
MEM[00587]: 3153
MEM[00660]: 7
MEM[00661]: 4
MEM[00662]: 7001
MEM[00663]: 7999
MEM[00664]: 3389
MEM[00665]: 3389
MEM[00666]: 1
MEM[00667]: 3390
MEM[00740]: 8
MEM[00741]: 4
MEM[00742]: 8001
MEM[00743]: 8999
MEM[00744]: 3638
MEM[00745]: 3638
MEM[00746]: 1
MEM[00747]: 3639
MEM[00820]: 9
MEM[00821]: 4
MEM[00822]: 9001
MEM[00823]: 9999
MEM[00824]: 3899
MEM[00825]: 3899
MEM[00826]: 1
MEM[00827]: 3900
MEM[00900]: 10
MEM[00901]: 4
MEM[00902]: 10001
MEM[00903]: 10999
MEM[00904]: 4174
MEM[00905]: 4174
MEM[00906]: 1
MEM[00907]: 4175
MEM[01001]: 10
MEM[01002]: 11
MEM[01003]: 12
MEM[01004]: 23
MEM[01005]: 34
MEM[01006]: 45
MEM[01007]: 56
MEM[01008]: 67
MEM[01009]: 78
MEM[01010]: 89
MEM[01011]: 90
MEM[01012]: 9
MEM[01013]: 1
MEM[01014]: 9
MEM[01015]: 9
MEM[01018]: 1003
MEM[01019]: 11
MEM[01020]: 1014
```



Key Observations from Final Memory Dump:

The following observations are based on the final memory dump provided by the simulator (see Appendix X for the full dump, if including).

- **CPU State and Global Counters:**
  - **Final Program Counter (PC - M[0])**: 125. This is the address of the HLT instruction within the GTU-OS code (specifically, the OS's halt instruction after its final statistics update when no active threads remain), indicating a normal system shutdown orchestrated by the OS.
  - **Final Stack Pointer (SP - M[1])**: 19999. This is the initial SP for the OS, suggesting that the OS's own stack usage was minimal or that its stack operations were perfectly balanced by the end.
  - **Total Instructions Executed (M[3])**: 13209. This reflects the total number of CPU cycles/instructions processed during the entire simulation.

    - **SYSCALL Saved PC (M[4])**: 3031. This value represents the return PC saved from the last thread that made a system call before the OS decided to halt. Given that Thread 3 (Fibonacci) instructions end at 3030 SYSCALL HLT, 3031 is the expected return address for that thread's final syscall.
- **OS Variables (Selected from M[21-99]):**
    - **current_thread_ID (M[21])**: Not explicitly shown in the non-zero dump, implying it is 0 (OS).
    - **num_active_threads (M[22])**: Not shown in non-zero dump, implying its final value is 0, which is correct as all user threads are expected to have halted.
    - **scheduler_state (M[23])**: 1. Indicates the scheduler was considered running/initialized.
    - **context_switch_counter (M[25])**: 66. The system performed 66 context switches.
    - **next_thread_to_check (M[27])**: 4. The scheduler's round-robin pointer stopped at Thread 4 in its last scan. That’s mean last thread, which is executed, is Thread 3.
    - OS global constants like max_threads (M[31]=11), thread_entry_size (M[32]=80), thread_base_offset (M[33]=100) are correctly preserved.
    - **last_thread_execution_start (M[34])**: 13104. These values to track the last Instruction Count of the last thread.
    - **instruction_count_of_last_executed_thread (M[35])**: 13112. These values relate to the final quantum of the last user thread that ran.
- **Thread Table (TT) Final States (Selected from M[100-979]):**
    - **Thread 0 (OS - M[100-179])**:
        - **M[101] (State)**: 4 (HALTED). This indicates that the OS, after performing its final shutdown procedures and ensuring all user threads were

terminated, also transitioned itself to a HALTED state before the CPU ceased execution.

- **M[102] (PC)**: 21. This remains the initial PC of the OS to start the OS.
- **M[103] (SP)**: 19999. Last reference of the memory size 20000.
- **M[105] (TT0 total_user_thread_IC)**: 1404. This value remains consistent, indicating that the total CPU instructions consumed by all user threads combined is 1404.
- **M[106] (TT0 total_executed_instructions)**: 11805. This represents the net instructions executed by the OS itself.

- **Thread 1 (Bubble Sort - M[180-259])**:
  - **M[181] (State)**: 4 (HALTED). Correct.
  - **M[182] (PC)**: 1052. Points to the instruction after its SYSCALL HLT.
  - **M[186] (total_executed_instructions)**: 1242.
- **Thread 2 (Linear Search - M[260-339])**:
  - **M[261] (State)**: 4 (HALTED). Correct.
  - **M[262] (PC)**: 2024. Points to the instruction after its SYSCALL HLT.
  - **M[266] (total_executed_instructions)**: 48.
- **Thread 3 (Fibonacci - M[340-419])**:
  - **M[341] (State)**: 4 (HALTED). Correct.
  - **M[342] (PC)**: 3031. Points to the instruction after its SYSCALL HLT.
  - **M[346] (total_executed_instructions)**: 107.
- **Threads 4-10 (Placeholder Halt Threads - e.g., M[420-499] for T4)**:
  - All show State 4 (HALTED).
  - Their PCs (e.g., M[422]=4001) point to the instruction after their SYSCALL HLT.

        - Each consumed 1 instruction (M[426]=1, M[506]=1, etc.), which is the SYSCALL HLT itself. This is correct.
- **Thread Data Areas & Results (M[1000+])**:
    - **Thread 1 (Bubble Sort - M[1001-1011])**:
        - The values from M[1002] to M[1011] are 11, 12, 23, 34, 45, 56, 67, 78, 89, 90. This is the correctly sorted version of the initial unsorted array (45, 23, 67, 12, 89, 34, 56, 78, 90, 11). The N value at M[1001] is 10. This confirms the sort algorithm worked.
    - **Thread 2 (Linear Search - M[2000+]):**
        - **M[2001] (RESULT)**: 2. The key 42 (at M[2021]) is found at index 2 within its array (M[2023]=10, M[2024]=25, M[2025]=42, ...). This is correct.
    - **Thread 3 (Fibonacci - M[3000+])**:
        - The console output (before the memory dump) shows the Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 (for N=10 at M[3001]).
        - **M[3003] (fib_prev2)**: 21.
        - **M[3004] (fib_prev1)**: 34.
        - **M[3005] (current_fib)**: 34.
        - **M[3002] (counter)**: 10. These final variable states are consistent with calculating 10 Fibonacci numbers.

Discussion: The final memory state from Debug Mode 0 strongly indicates the successful execution and normal termination of the GTU-OS and its managed threads.

1. The CPU halted in KERNEL mode with the PC at the OS's designated HLT instruction (125).

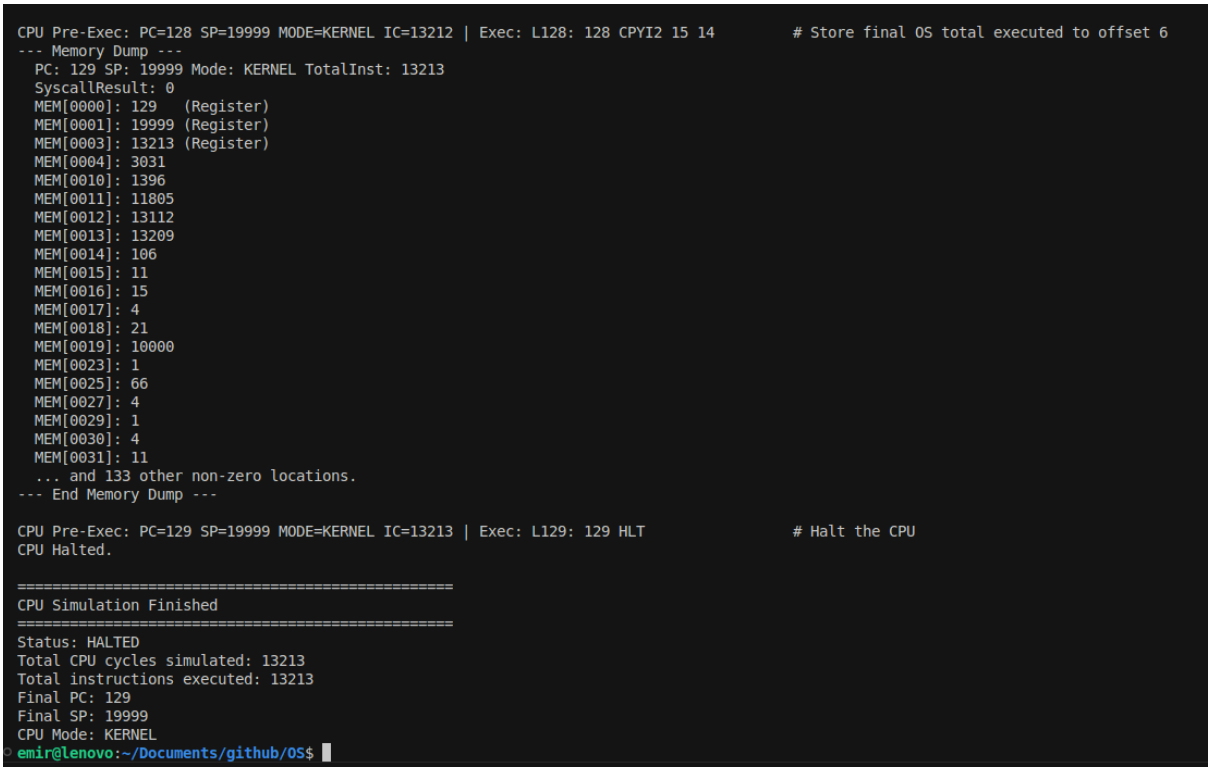
2. All user threads (1-10) correctly transitioned to the HALTED state, as evidenced by their Thread Table entries.
3. The individual algorithms executed by Threads 1, 2, and 3 produced the correct computational results, which are preserved in their respective data memory segments. The console output for Fibonacci also confirms its correctness.
4. The instruction accounting within the Thread Table appears consistent. The sum of total_executed_instructions for all user threads (1242 + 48 + 107 + 1*7 = 1404) matches the value stored by the OS in TT0[Offset 5] (M[00105]=1404).
5. Placeholder threads (4-10) also correctly executed their single SYSCALL HLT instruction and terminated.

Overall, the Debug Mode 0 output provides robust evidence that the implemented system functions as designed, from OS control flow and thread management to the execution of user-level application logic.

3. **Debug Mode 1: Per-Instruction Memory Trace**

- **Objective**: To observe detailed memory changes after every CPU instruction, offering a fine-grained view of dynamic CPU and OS operations.
- **Execution Overview**: Debug Mode 1 provides a verbose, continuous trace of all non-zero memory locations after each instruction. While the sheer volume of output makes a full inclusion impractical, specific sequences were examined to verify low-level mechanisms.
- **Key Observations and Illustrative Snippets**: The detailed trace confirmed the step-by-step execution of both OS and user thread instructions. Key OS operations, such as the initialization of Thread Table entries, the mechanics of context saving and loading during context switches, and the precise

moment of mode changes were observed to function as described in the Methodology section.

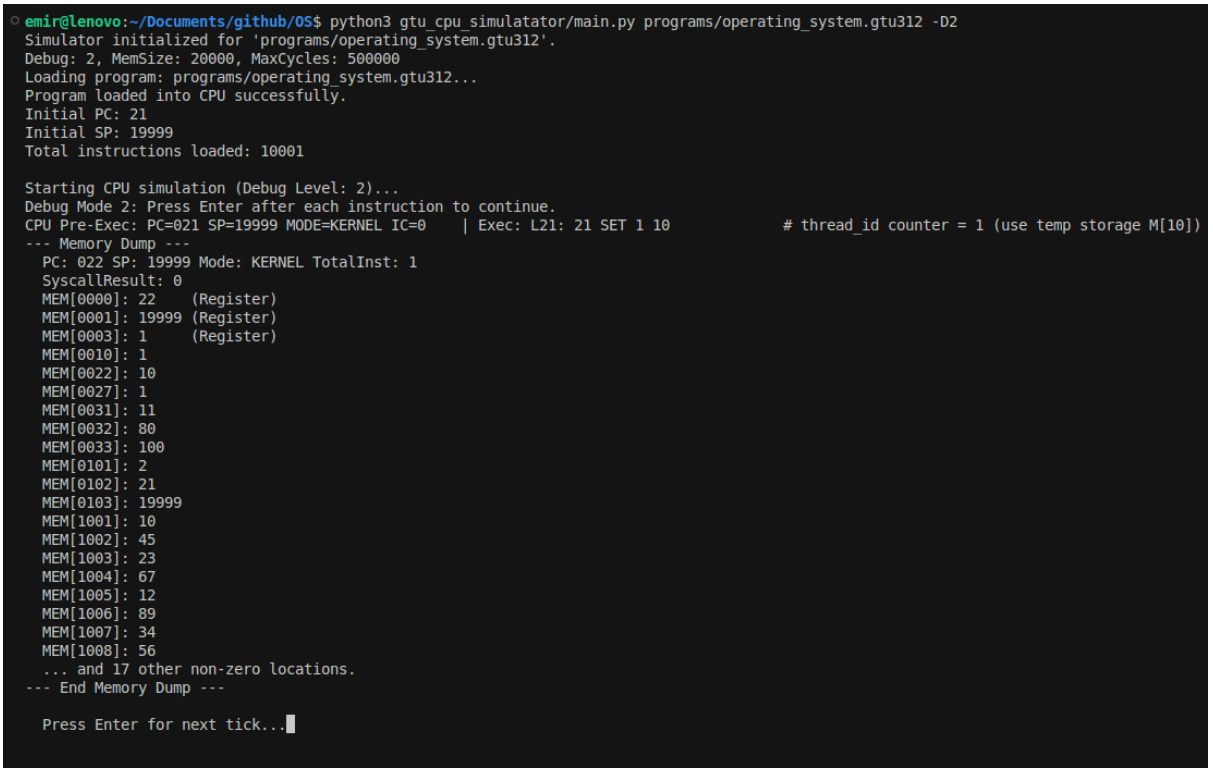


- **Overall System Outcome:** Despite the high level of detail, allowing the simulation to run to completion in Debug Mode 1 ultimately resulted in the same final memory state and thread termination statuses as observed and detailed in Debug Mode 0, confirming system-wide consistency.

4. Debug Mode 2: Step-by-Step Execution with Memory Trace

- **Objective**: To provide the same detailed per-instruction memory trace as Debug Mode 1, but with interactive stepping (requiring a keypress to advance), facilitating focused debugging and closer examination of specific execution phases.

- **Execution Overview**: This mode allowed for deliberate progression through the simulation, pausing after each instruction to inspect the memory state.
- **Key Observations and Illustrative Snippets**: The interactive nature of Debug Mode 2 was particularly useful for confirming the logic flow at decision points, such as conditional jumps (JIF) within user threads or the OS scheduler.



- **Overall System Outcome:** As with Debug Mode 1, allowing the simulation to proceed step-by-step to its conclusion in Debug Mode 2 led to the identical final halted state detailed under Debug Mode 0.

5. Debug Mode 3: Thread Table Dump on Critical Events

- **Objective**: To observe the OS's Thread Table at key events (USER instruction, after SYSCALL processing, and final halt) to validate high-level thread management.
- **Execution Overview**: The simulator provides formatted Thread Table dumps at these junctures.
- **Key Observations and Illustrative Snippets**: The Thread Table dumps consistently verified correct OS behavior:
    - **Initial Dispatch**: The first USER to Thread 1 showed T1 as RUNNING and other user threads READY, with the OS READY.
    - **State Transitions**: Subsequent dumps after SYSCALLs confirmed accurate transitions. For instance, after a SYSCALL PRN, the respective thread was marked BLOCKED with BlockDur=100. After a SYSCALL HLT, threads were marked HALTED. A SYSCALL YIELD resulted in the yielding thread becoming READY and another READY thread being dispatched as RUNNING.
    - **Context & Accounting**: Saved PCs, SPs, and instruction count fields (FirstExec, LastStart, TotalInst, LastIC) in the TT were updated correctly with each context switch and thread lifecycle event.
    - **Final State**: The final Thread Table dump showed all threads, including the OS, as HALTED, with TotalInst values consistent with those observed in the Debug Mode 0 final memory state.

```
emir@lenovo:~/Documents/github/OS$ python3 gtu_cpu_simulatator/main.py programs/operating_system.gtu312 -D3
Simulator initialized for 'programs/operating_system.gtu312'.
Debug: 3, MemSize: 20000, MaxCycles: 500000
Loading program: programs/operating_system.gtu312...
Program loaded into CPU successfully.
Initial PC: 21
Initial SP: 19999
Total instructions loaded: 10001

Starting CPU simulation (Debug Level: 3)...
CPU: Switched to USER mode. Jumping to PC=1000
--- Thread Table Dump ---
  OS : ID= 0 State=READY   PC=  21 SP=19999 FirstExec=   0 LastStart=   0 TotalInst= 897 LastIC=   0 BlockDur=  0
  T1 : ID= 1 State=RUNNING PC=1000 SP= 1999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T2 : ID= 2 State=READY   PC=2000 SP= 2999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T3 : ID= 3 State=READY   PC=3000 SP= 3999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T4 : ID= 4 State=READY   PC=4000 SP= 4999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T5 : ID= 5 State=READY   PC=5000 SP= 5999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T6 : ID= 6 State=READY   PC=6000 SP= 6999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T7 : ID= 7 State=READY   PC=7000 SP= 7999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T8 : ID= 8 State=READY   PC=8000 SP= 8999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T9 : ID= 9 State=READY   PC=9000 SP= 9999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T10: ID=10 State=READY   PC=10000 SP=10999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
--- End Thread Table Dump ---
CPU: SYSCALL from USER mode, switching to KERNEL mode.
SYSCALL.YIELD: Jumping to OS YIELD handler.
--- Thread Table Dump ---
  OS : ID= 0 State=READY   PC=  21 SP=19999 FirstExec=   0 LastStart=   0 TotalInst= 897 LastIC=   0 BlockDur=  0
  T1 : ID= 1 State=RUNNING PC=1000 SP= 1999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T2 : ID= 2 State=READY   PC=2000 SP= 2999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T3 : ID= 3 State=READY   PC=3000 SP= 3999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T4 : ID= 4 State=READY   PC=4000 SP= 4999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T5 : ID= 5 State=READY   PC=5000 SP= 5999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T6 : ID= 6 State=READY   PC=6000 SP= 6999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T7 : ID= 7 State=READY   PC=7000 SP= 7999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T8 : ID= 8 State=READY   PC=8000 SP= 8999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T9 : ID= 9 State=READY   PC=9000 SP= 9999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T10: ID=10 State=READY   PC=10000 SP=10999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
--- End Thread Table Dump ---
CPU: Switched to USER mode. Jumping to PC=2000
--- Thread Table Dump ---
  OS : ID= 0 State=READY   PC=  21 SP=19999 FirstExec=   0 LastStart=1168 TotalInst=1065 LastIC=   0 BlockDur=  0
  T1 : ID= 1 State=READY   PC=1041 SP= 1999 FirstExec= 929 LastStart= 929 TotalInst=1168 LastIC=2097 BlockDur=  0
  T2 : ID= 2 State=RUNNING PC=2000 SP= 2999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T3 : ID= 3 State=READY   PC=3000 SP= 3999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T4 : ID= 4 State=READY   PC=4000 SP= 4999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T5 : ID= 5 State=READY   PC=5000 SP= 5999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T6 : ID= 6 State=READY   PC=6000 SP= 6999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T7 : ID= 7 State=READY   PC=7000 SP= 7999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T8 : ID= 8 State=READY   PC=8000 SP= 8999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T9 : ID= 9 State=READY   PC=9000 SP= 9999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
  T10: ID=10 State=READY   PC=10000 SP=10999 FirstExec=   0 LastStart=   0 TotalInst=   0 LastIC=   0 BlockDur=  0
--- End Thread Table Dump ---
CPU: SYSCALL from USER mode, switching to KERNEL mode.
2
--- Thread Table Dump ---
```

... ...

```
  T10: ID=10 State=HALTED  PC=10001 SP=10999 FirstExec=4174 LastStart=4174 TotalInst=   1 LastIC=4175 BlockDur=  0
--- End Thread Table Dump ---
CPU: Switched to USER mode. Jumping to PC=3022
--- Thread Table Dump ---
  OS : ID= 0 State=READY   PC=  21 SP=19999 FirstExec=   0 LastStart=1396 TotalInst=11668 LastIC=   0 BlockDur=  0
  T1 : ID= 1 State=HALTED  PC=1052 SP= 1999 FirstExec= 929 LastStart=12867 TotalInst=1242 LastIC=12872 BlockDur=  0
  T2 : ID= 2 State=HALTED  PC=2024 SP= 2999 FirstExec=2269 LastStart=4576 TotalInst=  48 LastIC=4577 BlockDur=  0
  T3 : ID= 3 State=RUNNING PC=3022 SP= 3999 FirstExec=2507 LastStart=12272 TotalInst=  99 LastIC=12283 BlockDur=  0
  T4 : ID= 4 State=HALTED  PC=4001 SP= 4999 FirstExec=2714 LastStart=2714 TotalInst=   1 LastIC=2715 BlockDur=  0
  T5 : ID= 5 State=HALTED  PC=5001 SP= 5999 FirstExec=2927 LastStart=2927 TotalInst=   1 LastIC=2928 BlockDur=  0
  T6 : ID= 6 State=HALTED  PC=6001 SP= 6999 FirstExec=3152 LastStart=3152 TotalInst=   1 LastIC=3153 BlockDur=  0
  T7 : ID= 7 State=HALTED  PC=7001 SP= 7999 FirstExec=3389 LastStart=3389 TotalInst=   1 LastIC=3390 BlockDur=  0
  T8 : ID= 8 State=HALTED  PC=8001 SP= 8999 FirstExec=3638 LastStart=3638 TotalInst=   1 LastIC=3639 BlockDur=  0
  T9 : ID= 9 State=HALTED  PC=9001 SP= 9999 FirstExec=3899 LastStart=3899 TotalInst=   1 LastIC=3900 BlockDur=  0
  T10: ID=10 State=HALTED  PC=10001 SP=10999 FirstExec=4174 LastStart=4174 TotalInst=   1 LastIC=4175 BlockDur=  0
--- End Thread Table Dump ---
CPU: SYSCALL from USER mode, switching to KERNEL mode.
SYSCALL.HLT: Jumping to OS HLT handler.
--- Thread Table Dump ---
  OS : ID= 0 State=READY   PC=  21 SP=19999 FirstExec=   0 LastStart=1396 TotalInst=11668 LastIC=   0 BlockDur=  0
  T1 : ID= 1 State=HALTED  PC=1052 SP= 1999 FirstExec= 929 LastStart=12867 TotalInst=1242 LastIC=12872 BlockDur=  0
  T2 : ID= 2 State=HALTED  PC=2024 SP= 2999 FirstExec=2269 LastStart=4576 TotalInst=  48 LastIC=4577 BlockDur=  0
  T3 : ID= 3 State=RUNNING PC=3022 SP= 3999 FirstExec=2507 LastStart=12272 TotalInst=  99 LastIC=12283 BlockDur=  0
  T4 : ID= 4 State=HALTED  PC=4001 SP= 4999 FirstExec=2714 LastStart=2714 TotalInst=   1 LastIC=2715 BlockDur=  0
  T5 : ID= 5 State=HALTED  PC=5001 SP= 5999 FirstExec=2927 LastStart=2927 TotalInst=   1 LastIC=2928 BlockDur=  0
  T6 : ID= 6 State=HALTED  PC=6001 SP= 6999 FirstExec=3152 LastStart=3152 TotalInst=   1 LastIC=3153 BlockDur=  0
  T7 : ID= 7 State=HALTED  PC=7001 SP= 7999 FirstExec=3389 LastStart=3389 TotalInst=   1 LastIC=3390 BlockDur=  0
  T8 : ID= 8 State=HALTED  PC=8001 SP= 8999 FirstExec=3638 LastStart=3638 TotalInst=   1 LastIC=3639 BlockDur=  0
  T9 : ID= 9 State=HALTED  PC=9001 SP= 9999 FirstExec=3899 LastStart=3899 TotalInst=   1 LastIC=3900 BlockDur=  0
  T10: ID=10 State=HALTED  PC=10001 SP=10999 FirstExec=4174 LastStart=4174 TotalInst=   1 LastIC=4175 BlockDur=  0
--- End Thread Table Dump ---
CPU Halted.
--- Thread Table Dump ---
  OS : ID= 0 State=HALTED  PC=  21 SP=19999 FirstExec=   0 LastStart=1404 TotalInst=11805 LastIC=   0 BlockDur=  0
  T1 : ID= 1 State=HALTED  PC=1052 SP= 1999 FirstExec= 929 LastStart=12867 TotalInst=1242 LastIC=12872 BlockDur=  0
  T2 : ID= 2 State=HALTED  PC=2024 SP= 2999 FirstExec=2269 LastStart=4576 TotalInst=  48 LastIC=4577 BlockDur=  0
  T3 : ID= 3 State=HALTED  PC=3031 SP= 3999 FirstExec=2507 LastStart=13104 TotalInst= 107 LastIC=13112 BlockDur=  0
  T4 : ID= 4 State=HALTED  PC=4001 SP= 4999 FirstExec=2714 LastStart=2714 TotalInst=   1 LastIC=2715 BlockDur=  0
  T5 : ID= 5 State=HALTED  PC=5001 SP= 5999 FirstExec=2927 LastStart=2927 TotalInst=   1 LastIC=2928 BlockDur=  0
  T6 : ID= 6 State=HALTED  PC=6001 SP= 6999 FirstExec=3152 LastStart=3152 TotalInst=   1 LastIC=3153 BlockDur=  0
  T7 : ID= 7 State=HALTED  PC=7001 SP= 7999 FirstExec=3389 LastStart=3389 TotalInst=   1 LastIC=3390 BlockDur=  0
  T8 : ID= 8 State=HALTED  PC=8001 SP= 8999 FirstExec=3638 LastStart=3638 TotalInst=   1 LastIC=3639 BlockDur=  0
  T9 : ID= 9 State=HALTED  PC=9001 SP= 9999 FirstExec=3899 LastStart=3899 TotalInst=   1 LastIC=3900 BlockDur=  0
  T10: ID=10 State=HALTED  PC=10001 SP=10999 FirstExec=4174 LastStart=4174 TotalInst=   1 LastIC=4175 BlockDur=  0
--- End Thread Table Dump ---

==============================================
CPU Simulation Finished
==============================================
Status: HALTED
Total CPU cycles simulated: 13213
Total instructions executed: 13213
Final PC: 129
Final SP: 19999
CPU Mode: KERNEL
emir@lenovo:~/Documents/github/OS$
```

- **Overall System Outcome:** The Thread Table screenshots from Debug Mode 3 confirmed the integrity of the OS's thread state management, scheduling logic, and accounting, all leading to the same correct final system state detailed in Debug Mode 0.

4. Conclusion

This semester project, encompassing the design and simulation of the GTU-C312 hypothetical CPU and its bespoke operating system, GTU-OS, has provided a comprehensive and deeply engaging hands-on learning experience. The journey from conceptualizing the CPU's instruction set and memory architecture to implementing a multi-threaded operating system in its assembly language has been both challenging

and immensely rewarding. It required a practical application of theoretical knowledge in areas spanning computer architecture, operating system design, and low-level algorithmic implementation.

Successfully completing this project necessitated drawing extensively upon foundational operating systems principles, as covered in our Operating Systems course lectures and detailed in authoritative texts such as Andrew S. Tanenbaum's "Modern Operating Systems." Core concepts like process and thread management, CPU scheduling algorithms (specifically round-robin), context switching mechanisms, system call handling, memory organization, and thread state transitions were not merely studied but actively designed and coded. The constraints of the custom GTU-C312 instruction set demanded careful planning and a meticulous approach to implementing OS functionalities like the thread table, scheduler logic, and even the user threads performing sorting, searching, and Fibonacci calculations.

The simulation results, thoroughly examined across various debug modes, have validated the correctness of both the GTU-C312 CPU simulator and the GTU-OS. The system demonstrated reliable thread initialization, consistent cooperative multitasking via the round-robin scheduler, accurate context switching with state preservation, and proper handling of system calls including SYSCALL PRN, SYSCALL HLT, and SYSCALL YIELD. User threads successfully executed their designated algorithms, producing correct outputs and appropriately interacting with the OS. The final system state consistently showed a clean shutdown with all threads, including the OS itself, reaching a halted state, and instruction accounting remained consistent across different levels of observation.

Overall, this project has significantly strengthened both my theoretical understanding and practical skills in fundamental areas of computer science. The process of building a system from the ground up—from the CPU instruction execution to OS-level thread scheduling—has solidified my grasp of how these complex components interact. It has underscored the importance of meticulous design, rigorous testing, and the intricate dance between hardware and software that enables modern computing. The challenges overcome and the insights gained through this project will undoubtedly be invaluable in future academic and professional endeavors.

5. References

1.1. Tanenbaum, A. S., & Bos, H. (Year). Modern Operating Systems (5th ed.).

1.2. Gebze Technical University. (2025). CSE 312: Operating Systems - Lecture Recordings and Materials.

1.3. Round-robin scheduling from https://en.wikipedia.org/wiki/Round-robin_scheduling

1.4. OS İnformation from https://wiki.osdev.org/Expanded_Main_Page

1.5. Understand the concept from https://www.youtube.com/playlist?list=PLHh55M_Kq4OApWScZyPl5HhgsTJS9MZ6M